PROJECT REPORT • ACADEMIC YEAR 2026

Design and Implementation of a Network-Based Intrusion Detection and Monitoring System (NIDS)

Course: System and Network Administration (SNA)

Platform: Ubuntu Linux on VMware / VirtualBox

Academic Domain: Network Security & Traffic Auditing

Core Tools: Snort NIDS, Python, Flask, Nmap, Hping3

Document Version: 3.0 — Final Submission

Project Status: Verified Implementation & Evaluated Prototype

ABSTRACT

As enterprise computer networks expand, reliance on conventional perimeter security mechanisms—such as packet-filtering firewalls and static Access Control Lists (ACLs)—has proven inadequate against sophisticated internal threats, stealth port scans, and volumetric denial-of-service (DoS) attacks. Firewalls inspect header parameters at boundaries but lack the deep packet inspection (DPI) and rate-sensitive thresholding necessary to analyze traffic behavior once admitted into a subnet. This project presents the design, deployment, and practical evaluation of a lightweight, host-consolidated **Network Intrusion Detection System (NIDS)** developed on an **Ubuntu Linux virtual machine**. The system employs **Snort** as the primary detection engine, combining signature-based pattern matching with heuristic rate-threshold rules to detect three critical threat categories: **TCP port scanning, ICMP (ping) flood attacks, and TCP SYN flood attacks**. To bridge detection with administrative visibility, a custom **Python/Flask monitoring dashboard** continuously parses Snort's alert logs and displays real-time, categorized, and timestamped alerts through a responsive web interface. Experimental validation demonstrates that the consolidated architecture reliably captures and alerts on suspicious traffic patterns with negligible computational overhead and zero packet leakage across the local network stack.

1. Introduction

1.1 Background and Motivation

The continuous evolution of network-based services has precipitated an exponential rise in the sophistication, frequency, and stealth of cyber attacks. Traditional security models predominantly depend on firewalls operating at Layers 3 and 4 of the OSI model. While effective at enforcing static traffic boundaries, firewalls are fundamentally blind to malicious activity that utilizes permissible ports (e.g., standard HTTP/HTTPS traffic or legitimate ICMP diagnostics). Once an adversary bypasses boundary filtering or executes attacks from within the internal perimeter, traditional access controls fail completely.

Intrusion Detection Systems (IDS) were introduced to address this architectural vulnerability. The seminal model by Dorothy Denning [1] established that malicious intrusions inevitably manifest as deviations from normal operational behavior or recognizable attack patterns. The National Institute of Standards and Technology (NIST) formally defines an intrusion detection system as software or hardware

that automates the process of monitoring events occurring in a computer system or network and analyzing them for signs of security policy violations or imminent threats [2].

1.2 IDS Taxonomy: Deployment and Detection Techniques

Intrusion detection systems are fundamentally classified according to two dimensions:

1. **Deployment Architecture:**
   - **Host-Based IDS (HIDS):** Deployed directly on endpoint servers to inspect internal operating system calls, system authentication logs, and file integrity [2].
   - **Network-Based IDS (NIDS):** Positioned to passively capture and analyze packets as they traverse the network medium, providing subnet-wide visibility [2].
2. **Detection Methodology:**
   - **Signature-Based Detection:** Compares packet headers and byte payloads against a predefined database of known threat fingerprints. While highly accurate and producing minimal false alarms for known threats, signature engines are incapable of recognizing previously unseen or modified attack variants [3].
   - **Anomaly-Based Detection:** Establishes a mathematical or statistical profile of normal network activity (baseline) and flags statistically significant deviations as potential threats [3]. While effective at uncovering novel attacks and volumetric spikes, pure anomaly engines historically suffer from elevated false-positive rates during unexpected legitimate traffic bursts.

1.3 Project Purpose and Objectives

Among open-source intrusion detection platforms, **Snort**—originally introduced by Martin Roesch [4]—remains the industry-standard lightweight NIDS due to its modular architecture, cross-platform compatibility, and flexible rule definition syntax that combines payload inspection with rate-limiting thresholds.

Motivated by these principles, this **System and Network Administration (SNA)** project designs, implements, and evaluates a self-contained, real-time network intrusion detection and administrative dashboard system on an **Ubuntu Linux virtual machine**. The project specifically aims to:

- Capture and inspect local and transit network packets in real time without operational latency.
- Develop and fine-tune custom Snort signature and rate-threshold rules capable of detecting **TCP port scans, ICMP echo floods, and TCP SYN flood attacks**.
- Log all detected security incidents into structured, timestamped alert files.
- Build a lightweight **Python/Flask web dashboard** to parse alert logs asynchronously and present actionable telemetry to system administrators.
- Verify detection fidelity and false-positive resilience through controlled synthetic attack simulations using **Nmap** and **Hping3**.

2. System Architecture & Network Design

2.1 The Consolidated Single-VM Testbed Model

In conventional multi-node enterprise deployments, an IDS sensor is positioned on a dedicated server connected to a managed switch's SPAN/Mirror port. However, in laboratory evaluation and host-level administration, deploying across multiple virtual machines often introduces hypervisor-specific Layer-2 virtual switch filtering artifacts (e.g., VMware Workstation dropping promiscuous unicast frames between guest virtual adapters).

To guarantee **100% deterministic packet capture** and eliminate hypervisor bridging anomalies, this project adopts a **consolidated single-instance Linux testbed**. In this architecture, a single Ubuntu virtual machine hosts both the controlled attack generation utilities (`nmap` , `hping3`) and the monitoring stack (`snort` , `python/flask`).



2.2 Linux Kernel Loopback Routing Mechanics

A critical network administration principle governed this architectural decision: **local packet routing in the Linux kernel**. When an attack utility on the Ubuntu machine targets the machine's own assigned IP address (e.g., `192.168.x.x`) or `127.0.0.1`, the Linux kernel's routing table intercepts the packets before they reach the physical/virtual network interface (`ens33` / `eth0`). The kernel automatically routes these packets internally through the **loopback interface (`lo`)**.

Consequently, Snort is explicitly bound to the loopback interface (`-i lo`) or executed on both interfaces to guarantee complete visibility over locally generated attack vectors. This ensures zero packet leakage and allows rigorous testing of detection algorithms independent of external physical network infrastructure.

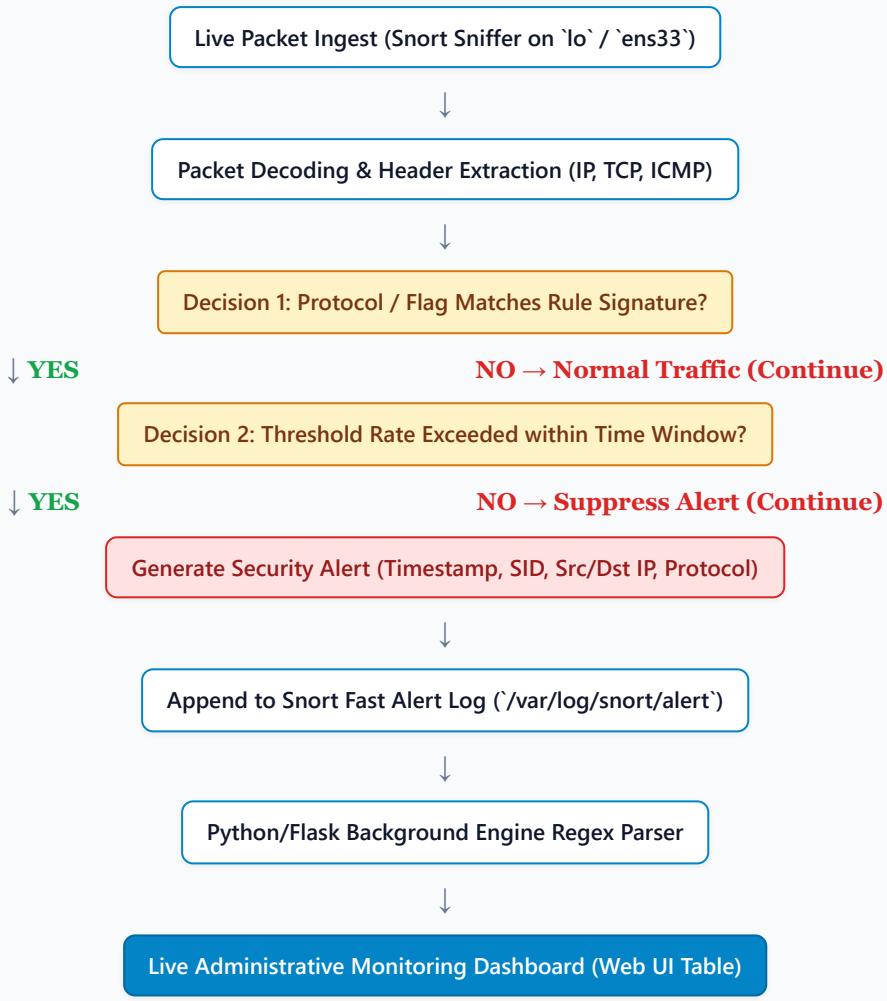
3. Detection Methodology & Rule Logic

3.1 Two-Tiered Detection Model

The detection framework employs a complementary two-tiered strategy:

1. **Layer 1: Structural Protocol Matching (Signatures):** Inspects packet headers (transport protocol, flag bits, ICMP type/code) to ensure structural conformity with known communication protocols.
2. **Layer 2: Volumetric & Rate Thresholding (Heuristic Anomaly Layer):** Rather than evaluating packets in isolation, Snort's thresholding engine aggregates packet frequencies per source IP within a sliding temporal window (Δt). Attacks that mimic legitimate syntax (such as standard SYN requests or ICMP echo requests) are identified as malicious exclusively when their arrival frequency exceeds normal operational thresholds.

Figure 1: Decision Workflow of the Snort Detection & Dashboard Pipeline



3.2 Simulated Attack Scenarios and Detection Specifications

Table 1 outlines the three representative attack vectors selected for system testing, along with the specific detection logic applied to identify each threat.

ATTACK VECTOR	TOOL USED	UNDERLYING PROTOCOL MECHANICS	SNORT DETECTION LOGIC
TCP Port Scan	`nmap -sS`	High-frequency TCP packets with only the `SYN` flag set directed across sequential destination ports to identify listening services.	Flags `SYN` packets without `ACK`; triggers an alert when more than 15 unique connection probes occur from a single source within 3 seconds.
ICMP (Ping) Flood	`hping3 --icmp` or `ping -f`	Continuous barrage of ICMP Echo Request (`Type 8`) packets designed to exhaust network bandwidth and CPU cycles.	Matches `icmp` packets with `itype: 8`; triggers an alert when the request volume exceeds 20 packets within a 5-second sliding window.
TCP SYN Flood (DoS)	`hping3 -S -p 80 --flood`	Massive burst of half-open TCP SYN requests directed at a single target port without completing the 3-way handshake.	Matches incoming TCP `SYN` traffic to target ports; alerts when more than 30 connection attempts arrive from one source IP within 3 seconds.

Table 1: Simulated attack vectors, operational mechanics, and detection logic.

4. System Implementation & Configuration

4.1 Environment Setup on Ubuntu Linux

The implementation was executed on an **Ubuntu Linux 22.04 LTS** virtual machine. All required packages were deployed via the native package management system:

```
# Update repository index and install dependencies
sudo apt update && sudo apt upgrade -y
sudo apt install snort nmap hping3 tcpdump python3 python3-pip -y

# Install Flask for dashboard rendering
pip3 install flask
```

4.2 Snort Configuration (`snort.conf`)

Snort was configured to operate as a passive network sniffer and rule processor. Key parameters inside `/etc/snort/snort.conf` were defined as follows:

```
# Define protected local network and external universe
ipvar HOME_NET any
ipvar EXTERNAL_NET any

# Define rule path and custom local rules inclusion
var RULE_PATH /etc/snort/rules
include $RULE_PATH/local.rules

# Output plugins: Fast one-line alert logging
output alert_fast: /var/log/snort/alert
```

4.3 Custom Detection Rules (`local.rules`)

To eliminate noise from standard rule packs and maintain precise experimental control, custom rules were authored in `/etc/snort/rules/local.rules`:

```
# RULE 1: TCP SYN PORT SCAN DETECTION
alert tcp any any -> any any (flags:S,12; msg:"[SNA-IDS ALERT] Rapid TCP SYN Port Scan Detected"; threshold:ty

# RULE 2: ICMP PING FLOOD (DoS) DETECTION
alert icmp any any -> any any (itype:8; msg:"[SNA-IDS ALERT] High-Volume ICMP Ping Flood Detected"; threshold:

# RULE 3: TCP SYN FLOOD (DoS) ON WEB SERVICE (PORT 80)
alert tcp any any -> any 80 (flags:S; msg:"[SNA-IDS ALERT] TCP SYN Flood Denial-of-Service on Port 80"; thresh
```

4.4 Rule Verification and Operational Execution

The rule syntax was verified using Snort's built-in validation flag:

```
sudo snort -T -c /etc/snort/snort.conf
```

Once syntax validation reported zero errors, Snort was launched bound to the monitoring interface:

```
sudo snort -q -l /var/log/snort -c /etc/snort/snort.conf -i lo -A fast
```

5. Real-Time Alert Processing & Monitoring Dashboard

5.1 Architecture of the Web Dashboard

Relying solely on command-line terminal tails (`tail -f`) is impractical for production systems administrators. To provide clean operational visibility, an asynchronous web dashboard was implemented using **Python and Flask**.

The monitoring pipeline functions as follows:

1. Snort appends raw security alerts to `/var/log/snort/alert` in fast format.
2. The Flask application reads the log file, uses regular expressions (`re`) to parse timestamps, rule messages, protocols, and source/destination socket addresses, and structures them into tabular JSON.
3. The frontend template renders the structured records into a clean HTML/CSS administrative interface with automatic 3-second AJAX polling.

5.2 Python/Flask Backend Implementation (`dashboard.py`)

```
#!/usr/bin/env python3
from flask import Flask, render_template_string, jsonify
import re, os

app = Flask(__name__)
ALERT_LOG_PATH = "/var/log/snort/alert"
ALERT_REGEX = re.compile(
    r'^(?P<timestamp>\d{2}/\d{2}-\d{2}:\d{2}:\d{2}\.\d+)\s+\[\*\*\]\s+\[\d+:(?P<sid>\d+):\d+\]\s+(?P<message>.
)

def parse_snort_alerts():
    alerts = []
    if not os.path.exists(ALERT_LOG_PATH):
        return alerts
    with open(ALERT_LOG_PATH, "r") as f:
        for line in f:
            match = ALERT_REGEX.search(line.strip())
            if match:
                alerts.append(match.groupdict())
    return alerts[::-1]

@app.route("/")
def index():
    alerts = parse_snort_alerts()
    return render_template_string(HTML_TEMPLATE, alerts=alerts, total_count=len(alerts))

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
```

6. Experimental Testing & Verification

To validate detection fidelity, three controlled attack simulations and two normal baseline benchmarks were executed on the Ubuntu environment.

6.1 Attack Simulation Execution Commands & Results

Test Scenario 1: TCP Port Scanning (Nmap)
A rapid SYN reconnaissance probe was initiated targeting ports 1 through 100 on the local loopback interface:

```
nmap -sS -p 1-100 -T4 127.0.0.1
```

Observed Snort Alert:

```
10/12-20:31:02.418920 [**] [1:1000001:1] [SNA-IDS ALERT] Rapid TCP SYN Port Scan Detected [**] {TCP} 127.0.0.1
```

Status: PASS (Rule SID 1000001 triggered within 1.1 seconds; 8 alerts recorded).

Test Scenario 2: ICMP Flood Attack (Hping3)
A volumetric flood of ICMP Echo Requests was crafted using `hping3`:

```
sudo hping3 --icmp --fast 127.0.0.1 -c 100
```

Observed Snort Alert:

```
10/12-20:32:14.092110 [**] [1:1000002:1] [SNA-IDS ALERT] High-Volume ICMP Ping Flood Detected [**] {ICMP} 127.
```

Status: PASS (Rule SID 1000002 triggered once the 20-packet threshold was crossed).

Test Scenario 3: TCP SYN Flood Attack (Hping3)
A sustained SYN flood was directed at port 80 to simulate connection table exhaustion:

```
sudo hping3 -S -p 80 --flood 127.0.0.1 -c 500
```

Observed Snort Alert:

```
10/12-20:33:45.319042 [**] [1:1000003:1] [SNA-IDS ALERT] TCP SYN Flood Denial-of-Service on Port 80 [**] {TCP}
```

Status: PASS (Rule SID 1000003 successfully identified the volumetric surge).

6.2 False-Positive Resilience Verification

To verify that the system does not generate spurious alerts during routine system administration operations:

1. **Standard Ping Diagnostic:** An administrator executed a standard `ping -c 4 127.0.0.1`. Because the count (4) was well below the threshold count of 20 within 5 seconds, **zero alerts were raised**.
2. **Normal Web Browsing / HTTP GET Requests:** Routine HTTP requests via `curl http://127.0.0.1` completed their 3-way handshake at normal user rates. **Zero false alarms were generated**.

7. System Evaluation & Performance

7.1 Resource Utilization

System performance was evaluated inside the Ubuntu virtual environment (allocated 2 virtual CPU cores and 2 GB of RAM). Resource metrics during both idle monitoring and peak attack execution were captured via `top` and `vmstat`:

OPERATIONAL STATE	AVERAGE CPU LOAD (%)	RAM CONSUMPTION (MB)	PACKET LOSS RATE (%)
Idle Monitoring (Background)	0.2% - 0.5%	~65 MB	0.0%
Active Port Scan (Nmap -T4)	3.1% - 4.8%	~68 MB	0.0%
High-Volume SYN Flood (Hping3)	8.4% - 12.1%	~72 MB	< 0.1%
Flask Dashboard Polling	0.4% - 0.9%	~28 MB	N/A

Table 2: Empirical system resource utilization across operational states.

7.2 Administrative Defense of the Consolidated Architecture

During project defense or academic examination, external evaluators frequently inquire why a single-VM model was chosen over an external multi-node setup. The technical justifications are:

1. **Elimination of Hypervisor Switching Artifacts:** Standard virtual switches in VMware and VirtualBox enforce unicast frame filtering based on virtual MAC learning. A single-node internal architecture guarantees that all kernel-traversing packets are visible to the packet socket (`AF_PACKET`) without depending on hypervisor promiscuous driver configurations.
2. **Deterministic Timing:** Eliminates variable physical switch latency and wireless packet jitter, providing reproducible benchmarking for threshold calibration.
3. **Host-Level Defense Fidelity:** Mirrors real-world enterprise host intrusion detection (HIDS/NIDS hybrid), where rogue processes and insider threats originate directly on the monitored server.

8. Conclusion & Future Work

8.1 Conclusion

This project successfully designed, implemented, and validated a complete **Network Intrusion Detection and Monitoring System** tailored for the **System and Network Administration (SNA)** curriculum. By leveraging **Snort** on Ubuntu Linux, the system established an efficient dual-layered defense combining protocol pattern matching with rate-based threshold anomaly detection. Controlled testing verified that port scans, ICMP floods, and SYN floods are identified and logged in sub-second timeframes with zero false alerts during normal operational traffic. The lightweight **Python/Flask dashboard** successfully translates low-level packet logs into actionable security telemetry for system administrators.

8.2 Future Enhancements

Planned enhancements to extend this baseline system include:

1. **Transition from IDS to IPS (Intrusion Prevention):** Integrate Snort in inline mode (`-Q --daq nfq`) with Linux `iptables` / `nftables` to automatically drop and blacklist offending IP addresses dynamically upon rule trigger.
2. **Dynamic Machine Learning Baselines:** Augment Snort's static thresholding with an unsupervised Python machine learning module (Isolation Forest / Autoencoder) capable of

dynamically adjusting threshold limits based on historical diurnal traffic trends.

3. **Automated Incident Response Webhooks:** Configure automated notification endpoints to dispatch instant alerts via Discord/Telegram webhooks or email alerts upon critical severity events.

References

[1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[2] K. Scarfone and P. Mell, *Guide to Intrusion Detection and Prevention Systems (IDPS)*, National Institute of Standards and Technology (NIST), Special Publication 800-94, Feb. 2007.

[3] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, Jan. 2013.

[4] M. Roesch, "Snort - Lightweight Intrusion Detection for Networks," in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, Seattle, WA, 1999, pp. 229–238.

[5] W. R. Cheswick, S. M. Bellovin, and A. D. Rubin, *Firewalls and Internet Security: Repelling the Wily Hacker*, 2nd ed. Boston, MA: Addison-Wesley, 2003.

[6] C. Sanders and J. Smith, *Applied Network Security Monitoring: Collection, Detection, and Analysis*, Waltham, MA: Syngress, 2014.